

Nombre(s): Oliver Alexander

Apellidos: Mitma Quino

Código: 17200175

Fecha: 20/06/2026

1 Examen de Desarrollo de Sistemas Móviles - Nivel Básico - Intermedio

Información General

Puntaje Total: 20 puntos**Duración:** 2 horas + 10 min para entrega**Modalidad:** Preguntas Abiertas y Desarrollo de Código.

Instrucciones: Resuelva el examen de forma individual y a mano. Al finalizar, digitalice todas las páginas en un único archivo PDF denominado prueba1.pdf y súbalo al repositorio: github.com/illarek-lab/mobile_pruebas

Dentro del repositorio cree un branch C<codigo_estudiante> con un directorio con el formato: C<codigo_estudiante> y coloque en su interior el archivo: prueba1.pdf y realice su PR. **Ejemplo:** C20251234/prueba1.pdf

2 I. Análisis de Arquitectura y Concurrencia (8 puntos)**Valor: 2 pts c/u****1. El Patrón Repositorio como SSOT**

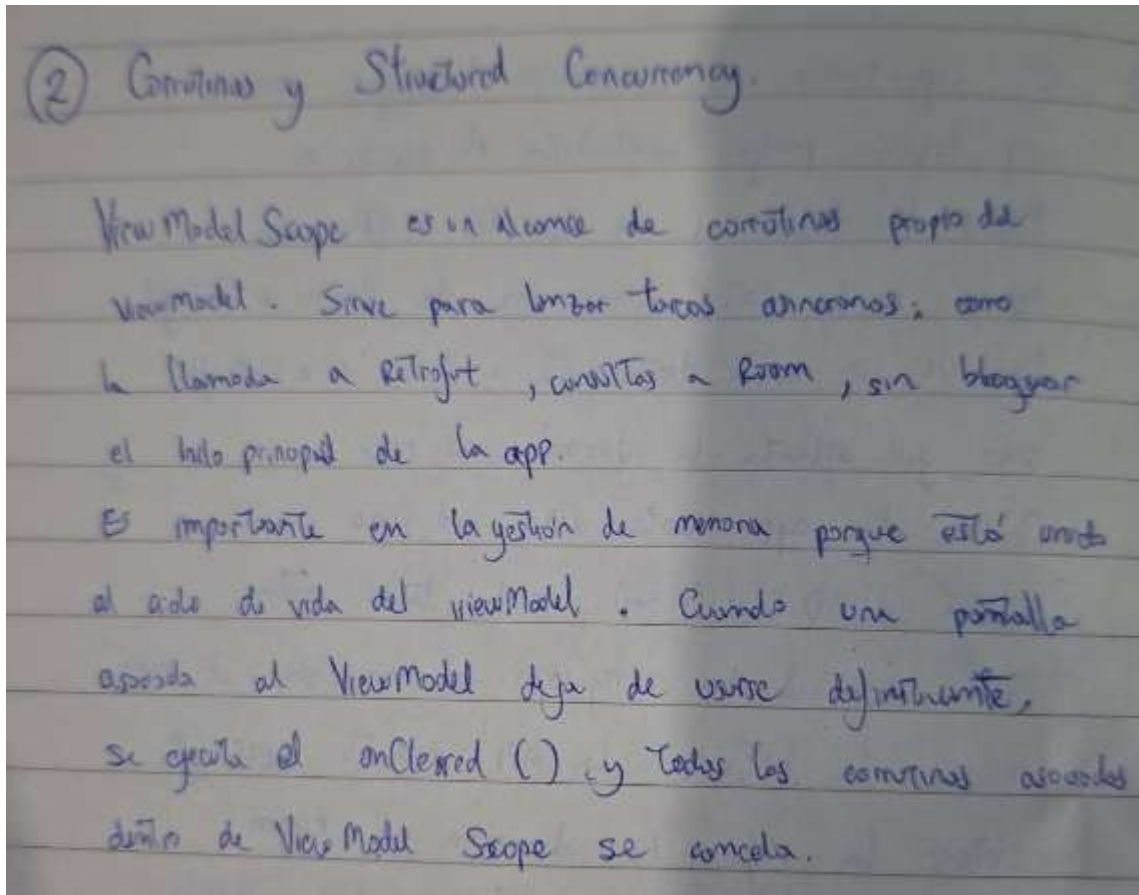
Explique por qué el Repositorio se considera la “Única Fuente de Verdad” (SSOT) y una ‘fachada’ en la arquitectura. Detalle cómo coordina los datos entre la base de datos local (Room) y la API remota (Retrofit) para que la UI no sepa de dónde vienen los datos.

① El repositorio se considera la single source of truth porque centraliza el acceso a todos los datos y de acuerdo a esto se define si los datos provienen de Room o Retrofit. La UI no accede directamente a Room ni a Retrofit sino que solicita la información al repo.

② Cuando la app necesita datos, el repo consulta primero la BD local (room) - si los datos existen, están desactualizados o requieren sincronización, el repo realiza la petición a la API mediante Retrofit obtiene la info y la almacena en Room. Por último UI observa los datos desde Room. Por ello recibe los datos automáticamente sin saber de donde provienen del local o internet.

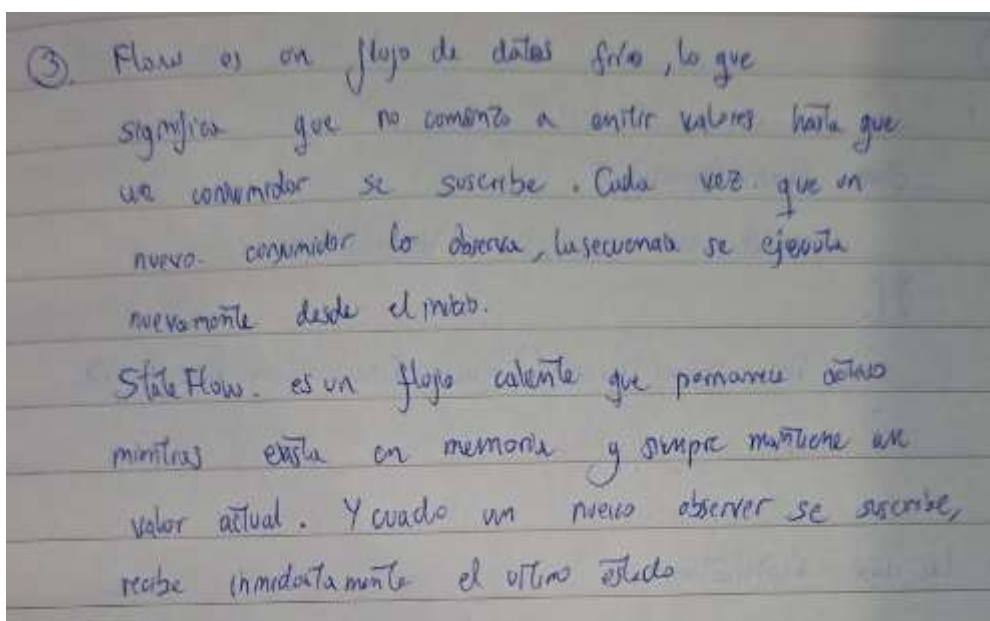
2. Corrutinas y Structured Concurrency

Defina qué es el `viewModelScope` y por qué es fundamental para la gestión de memoria. Explique qué sucede con una petición de red iniciada en este scope si el usuario decide cerrar la pantalla antes de recibir la respuesta.



3. Flujos Reactivos (Cold vs. Hot)

Diferencie técnicamente un `Flow` (frío) de un `StateFlow` (caliente). ¿Por qué para el manejo de estados de la interfaz de usuario se prefiere `StateFlow` sobre un `Flow` convencional?



4. Inyección de Dependencias (DI) Manual

Describa el mecanismo de “DI Manual” utilizado en los laboratorios mediante la clase `Application`. Explique para qué sirve la palabra reservada `by lazy` en la inicialización de los objetos de infraestructura (como la BD o el Repositorio).

④. La Inyección de Dependencias consiste en crear y proporcionar las dependencias sin utilizar frameworks. El mecanismo se implementa mediante una clase que hereda de `Application`, que se encarga de crear y almacenar instancias compartidas de objetos como la BD, DAO, repos.

3 II. Sección de Completar (4 puntos)

Instrucción: Escriba el término técnico exacto que completa la oración basándose en el texto de las sesiones.

1. El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo **Flow**.

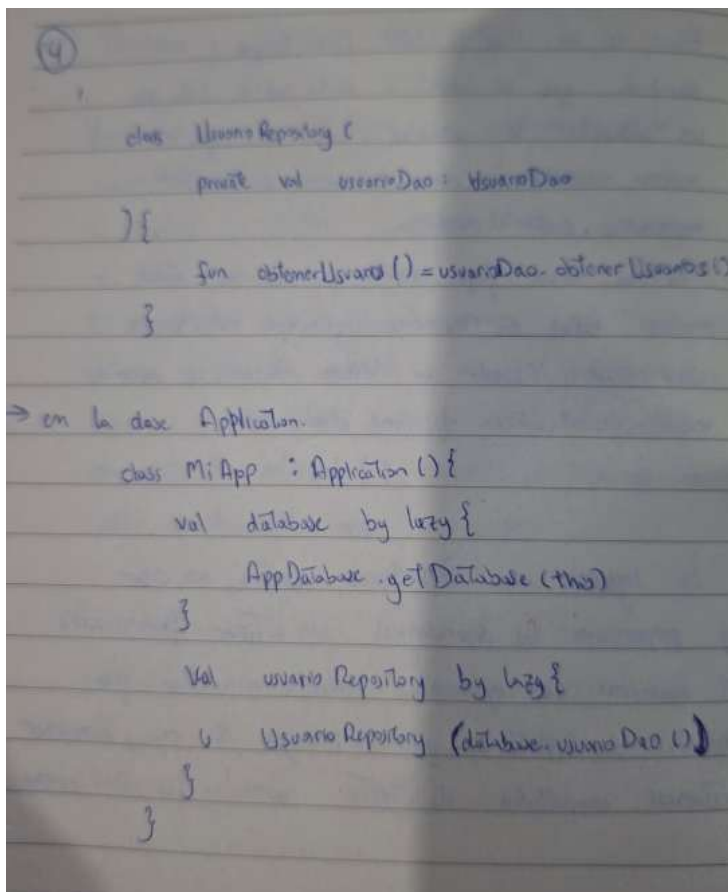
2. El operador *Dispatchers.Default* permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas de CPU como el parseo masivo de datos.
3. Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo *location*.
4. La función *applicationContext* de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast".
5. El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama *suspendCoroutine*.
6. El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente *AndroidManifest.xml*
7. En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina *HAL*.
8. El operador de Flow que permite combinar múltiples fuentes (como GPS y Sensores) para emitir un estado unificado es *combine*.

4 III. Desarrollo de Código y Lógica Personalizada (5 puntos)

Instrucción: Escriba el código en Kotlin siguiendo las convenciones de PascalCase para clases y camelCase para variables.

1. (2 pts) Repositorio y DI

Escriba el código de una clase `UsuarioRepository` que reciba por constructor un DAO. Luego, muestre cómo se instanciaría este repositorio dentro de una clase que extienda de `Application` usando el patrón `by lazy`.



```

④
class UsuarioRepository {
    private val usuarioDao: UsuarioDao
}

fun obtenerUsuario() = usuarioDao.obtenerUsuarios()

→ en la clase Application.
class MiApp : Application() {
    val database by lazy {
        AppDatabase.getDatabase(this)
    }

    val usuarioRepository by lazy {
        UsuarioRepository(database.usuarioDao())
    }
}
  
```

2. (3 pts) Lógica Dinámica de Identidad

- a) Cree una data class `Usuario` con los atributos: `nombre (String)`, `apellidos (String)` y `edad (Int?)`.
- b) Cree un objeto con sus datos reales.
- c) Escriba una lógica que calcule el “Puntaje de Identidad”: la suma del número de letras de su nombre + el número de letras de sus apellidos.
- d) Escriba el código para imprimir el mensaje:

Usuario: [Nombre Completo], tu puntaje de identidad es: [Resultado]

““

- e) Use el operador Elvis (`?:`) para asegurar que el puntaje sea 0 si los datos fuesen nulos. ““

```

2.
data class Usuario {
    val nombre: String?,
    val apellidos: String?,
    val edad: Int?

    fun mostrar() {
        val usuario = Usuario(
            nombre = "Oliver",
            apellidos = "Molina Garcia",
            edad = 29
        )

        val porcentajeIdentidad =
            (usuario.nombre?.replace(" ", "")?.length ?: 0) +
            (usuario.apellidos?.replace(" ", "")?.length ?: 0)

        println(
            "Usuario: $ {usuario.nombre} $ {usuario.apellidos}, tu  

            porcentaje de identidad es: $ porcentajeIdentidad"
        )
    }
}

```

5 IV. Arquitectura del AOSP Completa (3 puntos)

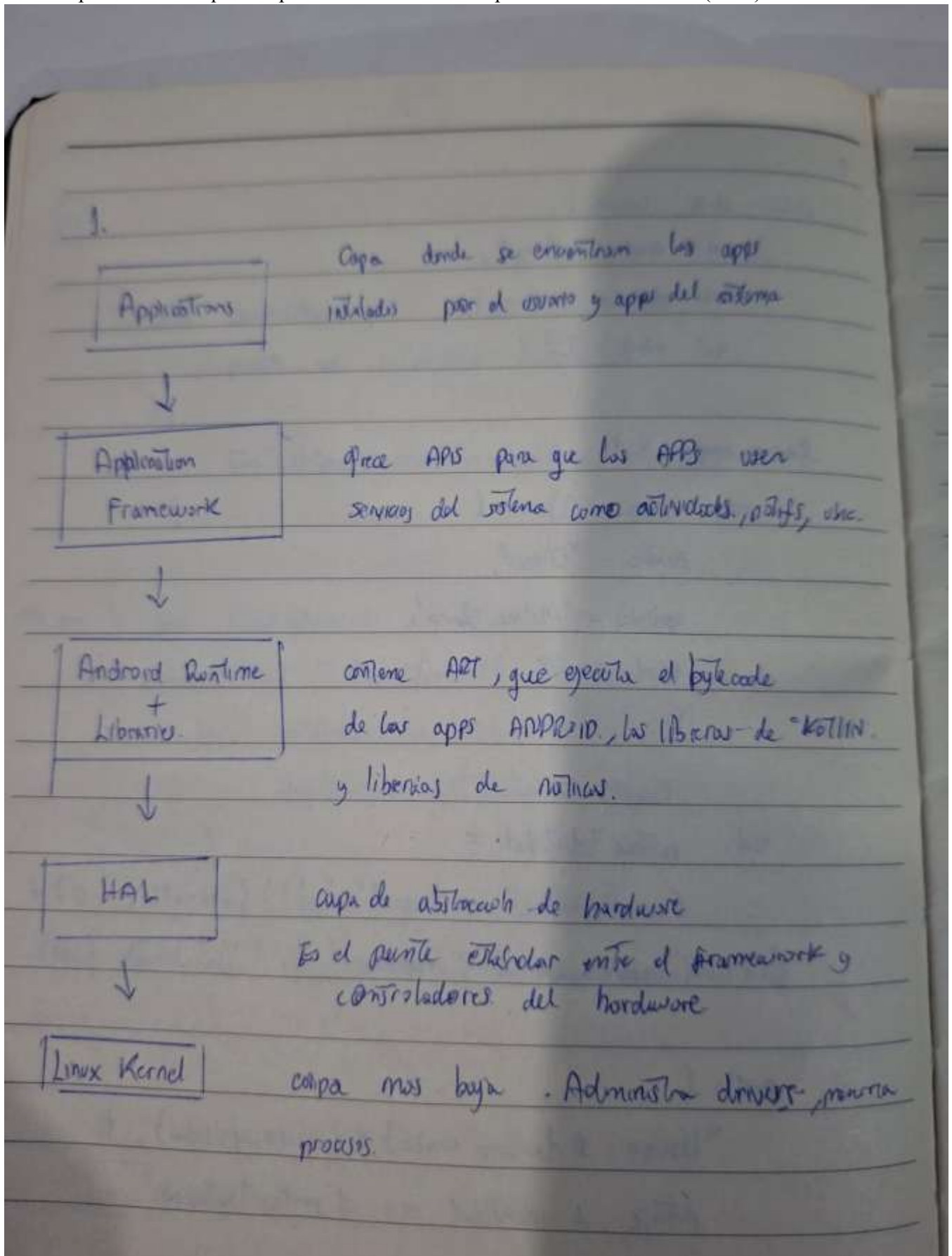
Instrucción: La comprensión de la pila de software es vital para el desarrollo nativo.

1. Gráfico (1.5 pts)

Dibuje el diagrama de las 5 capas de la arquitectura del Android Open Source Project (AOSP) en el orden jerárquico correcto.

2. Identificación Total (1.5 pts)

Nombre y describa brevemente la función de cada una de las 5 capas que graficó, explicando específicamente qué componentes residen en la capa de Android Runtime (ART).



Nota para el Estudiante

Se evaluará la precisión terminológica (por ejemplo, no confundir `Thread` con `Coroutine`) y el respeto a la jerarquía de capas de la arquitectura Android moderna.